

Spec-Driven Development Competitive Analysis

Drydock Against the Practical Field and the Wider Field

Scored feature comparison with explicit winners

Ed Barlow Web Cloud Studio August 5, 2026



Spec-Driven Development Competitive Analysis

Field	Value
Analysis date	2026-08-05 — supersedes the 2026-07-17 edition
Subject	Drydock scored against the four products it is most often compared to, then against the wider eleven-product field
Method	Drydock scored against docs/Drydock_Specification.md as implemented. Every other product scored from published primary documentation. No competitor was executed as part of this analysis; scores are documentation-derived engineering judgment, not measurements.
Bias disclosure	Part One's rows are drawn from the problem Drydock solves — governed delivery of a large system from an existing written specification. That biases the aggregate in Drydock's favor by construction. The honest reads are Section 9, the scenario table, and the closing list of losses.

Scoring Legend

Cell	Score	Meaning
●	5	Best in field; deterministic, enforced, and a defining strength
●	4	Strong first-class capability
●	3	Present and usable; advisory rather than enforced
●	2	Partial, manual, or emergent from adjacent features
●	1	Nominal or documentation-only
●	0	Absent

Part One — The Practical Comparison

Five products: the four Drydock is actually evaluated against, plus the baseline that everything must beat.

Key	Product	What it is	Cost model
DD	Drydock	A build system for specifications. Compiles a Blueprint into a dependency graph and executes it under test gates.	Proprietary; runs on your LLM subscription
SK	GitHub Spec Kit	A prompt and template kit turning an idea into spec → plan → tasks, then handing tasks to your agent.	Open source
KI	Kiro (AWS)	An IDE with a spec-first mode: requirements, design, tasks, agent hooks.	Proprietary IDE, paid tiers
BM	BMAD-METHOD	A multi-agent role framework — analyst, PM, architect, scrum master, dev, QA — driving elicitation and story delivery.	Open source
CM	Plain CLAUDE.md	One rules file and an agent. The honest baseline.	Free

The critical distinction. Drydock does not write your specification. Authoring is an explicit non-goal — `drydock import` assumes you arrive with source material. Spec Kit, Kiro, and BMAD are primarily *specification authoring* front-ends whose build side is "hand the tasks to an agent." Drydock is the opposite shape: no front-end, an industrial back-end. Treating them as one category is the most common error in this space.

1 — Specification Authoring and Ingestion

#	Capability	DD	SK	KI	BM	CM	Winner
1.1	Creates a specification from a blank page (elicitation, brainstorming, PRD)	● 0	● 4	● 4	● 5	● 2	BMAD
1.2	Ingests arbitrary pre-existing source material as the input of record	● 5	● 1	● 2	● 3	● 2	Drydock
1.3	Typed specification files with prescribed roles and validation	● 5	● 3	● 3	● 3	● 1	Drydock
1.4	Specification quality audit against the raw sources	● 4	● 0	● 1	● 2	● 0	Drydock
1.5	Specification remains authoritative after code exists	● 5	● 2	● 3	● 2	● 1	Drydock
	Section average	3.8	2.0	2.6	3.0	1.2	

Read. BMAD wins the row that matters to a team with nothing written down, and it wins outright — its analyst/PM/architect chain is better at extracting a coherent product definition from a founder's head than anything Drydock has, because Drydock has nothing here at all. A 3.8 average in a section whose headline row scores zero is the pattern for this entire document: Drydock is strong everywhere downstream of "the specification exists."

2 — Planning and Decomposition

#	Capability	DD	SK	KI	BM	CM	Winner
2.1	Dependency-graph build plan with a computed runnable frontier	● 5	● 1	● 1	● 2	● 0	Drydock
2.2	Story decomposition with stable IDs, states, and depends: edges	● 5	● 3	● 4	● 4	● 0	Drydock
2.3	Effort estimate expressed as token cost, not vibes	● 5	● 0	● 0	● 2	● 0	Drydock
2.4	Blocker gate that halts planning until the human answers	● 5	● 1	● 2	● 3	● 0	Drydock
2.5	Human-editable plan: reorder, regroup, rename, split	● 5	● 2	● 3	● 2	● 1	Drydock
	Section average	5.0	1.4	2.0	2.6	0.2	

Read. Uncontested. `MANIFEST.md` is a graph database carrying build state, provenance hashes, and grouping; the others emit an ordered task list. Row 2.3 is the one nobody else attempts — pricing story points in tokens makes context an explicitly budgeted resource.

3 — Build Execution

#	Capability	DD	SK	KI	BM	CM	Winner
3.1	Executes the plan autonomously, block by block, to working software	● 5	● 3	● 4	● 4	● 3	Drydock
3.2	Per-step context stacking — only the specs that step needs	● 5	● 2	● 2	● 4	● 1	Drydock
3.3	Automatic repair loop on failure with progress-based continuation	● 5	● 0	● 1	● 2	● 1	Drydock
3.4	Model escalation on the final repair attempt	● 5	● 0	● 0	● 1	● 0	Drydock
3.5	Parallel or worktree-isolated execution of independent work	● 0	● 0	● 1	● 2	● 1	BMAD
3.6	Resume in place after failure with partial work and failing checks intact	● 5	● 1	● 2	● 2	● 1	Drydock
	Section average	4.2	1.0	1.7	2.5	1.2	

Read. Row 3.3 is the feature Drydock is built around, and nobody else implements it as a loop with a *termination condition*: repair continues only while the passing set grows without regression, or while a per-criterion case tally improves without another regressing. That is a convergence rule, not a retry counter.

Row 3.5 is a real loss. Drydock computes the exact dependency graph that makes safe parallelism trivial, then builds strictly serially. BMAD wins the row by two points on a five-point scale — nobody here is good at it — but Drydock is the one leaving free throughput on the table.

4 — Verification and Test Integrity

#	Capability	DD	SK	KI	BM	CM	Winner
4.1	Enforced TDD gate — acceptance criteria exist and fail before implementation	● 4	● 1	● 2	● 3	● 2	Drydock
4.2	Acceptance verified deterministically by executing it, with no LLM in the loop	● 5	● 0	● 1	● 1	● 0	Drydock
4.3	Proof integrity — vacuous or tautological proofs demoted to UNVERIFIED	● 5	● 0	● 0	● 0	● 0	Drydock
4.4	EARS-notation project acceptance criteria with stable IDs	● 5	● 0	● 3	● 0	● 0	Drydock
4.5	Guardrails as absolute prohibitions behind a hard gate	● 5	● 1	● 0	● 1	● 2	Drydock
4.6	Release verdict with enumerated blockers	● 5	● 0	● 1	● 2	● 0	Drydock
	Section average	4.8	0.3	1.2	1.2	0.7	

Read. The widest margin in the document and the least contestable. Every other product here lets the agent that wrote the code decide whether the code is correct. Drydock separates the two: `drydock score ac` runs each criterion as a process and reports PASS / FAIL / UNVERIFIED, and static analysis demotes an empty body, a constant assertion, or a self-comparison rather than counting it as proof.

The deduction on 4.1 is deliberate: the RED→GREEN sequence is enforced at the gate and vacuous criteria are demoted, but Drydock does not hash-lock the proof artifact before implementation, so a determined agent can still shape its own target. Four, not five.

5 — Change Management

#	Capability	DD	SK	KI	BM	CM	Winner
5.1	Drift detection on applied specs via content hash plus commit provenance	● 5	● 0	● 1	● 0	● 0	Drydock
5.2	Sealed foundational specifications requiring an explicit change ticket	● 5	● 0	● 0	● 1	● 0	Drydock
5.3	Incremental replan scoped to impacted work only	● 5	● 1	● 2	● 2	● 0	Drydock
5.4	Change the specification, then rebuild — the normal path, not a restart	● 5	● 2	● 3	● 3	● 1	Drydock
5.5	Regression reopens a previously verified sibling story	● 4	● 0	● 0	● 0	● 0	Drydock
	Section average	4.8	0.6	1.2	1.2	0.2	

Read. This is where the other four stop being comparable products. They are strongest on the first pass and weakest on the second; their answer to "the specification changed" is largely "run it again and reconcile by hand." Drydock closes the loop: `drydock refit` detects changed Blueprints by checksum and commit, requires a ticket to touch `ARCHITECTURE.md`, `DATABASE.md`, or `UI-GENERAL.md`, remaps the Manifest, and

resets exactly the work that must be rebuilt. A build blocks outright when applied specifications have drifted. That is change control, not iteration.

6 — Context and Cost Economics

#	Capability	DD	SK	KI	BM	CM	Winner
6.1	Compaction of rules and specifications to a callable surface	● 5	● 0	● 0	● 3	● 1	Drydock
6.2	Automatic full-versus-compact selection driven by the build graph	● 5	● 0	● 0	● 0	● 0	Drydock
6.3	Runs on a subscription CLI with no per-token API spend	● 5	● 3	● 0	● 3	● 5	Tie: Drydock / CLAUDE.md
6.4	Large project delivered with a cheaper model	● 4	● 2	● 2	● 3	● 1	Drydock
	Section average	4.8	1.3	0.5	2.3	1.8	

Read. Kiro's 0.5 is structural: an IDE with metered inference, whose cost curve on a large project is its principal adoption objection. Drydock's position — compaction, graph-driven context selection, token-priced stories, subscription-only execution — is the second moat after verification. Note the tie on 6.3: a plain CLAUDE.md is exactly as cheap.

7 — Governance and Enterprise Standards

#	Capability	DD	SK	KI	BM	CM	Winner
7.1	Injected enterprise layer: business rules, stack rules, branding	● 5	● 2	● 3	● 2	● 4	Drydock
7.2	Persistent intent injected at every process stage	● 5	● 3	● 3	● 3	● 4	Drydock
7.3	Reviewable build evidence written per step	● 5	● 1	● 2	● 2	● 0	Drydock
7.4	Dependency legitimacy guard on generated package changes	● 4	● 0	● 0	● 0	● 1	Drydock
	Section average	4.8	1.5	2.0	1.8	2.3	

Read. CLAUDE.md places second here and deserves to — rows 7.1 and 7.2 are literally what a rules file does, at zero setup cost. Drydock's advantage is that Rigging is portfolio-scoped, versioned, compacted, and injected by the build rather than hoped for, and that the build writes evidence a reviewer can audit afterward. If the requirement is only "follow our house style," a CLAUDE.md delivers most of this section for a fraction of the effort.

8 — Human Review Surface

#	Capability	DD	SK	KI	BM	CM	Winner
8.1	Dedicated review console for plan, evidence, and decisions	● 5	● 0	● 4	● 1	● 0	Drydock

#	Capability	DD	SK	KI	BM	CM	Winner
8.2	Questionnaires whose answers persist into later commands	● 5	● 1	● 2	● 3	● 0	Drydock
8.3	Board view of plan state	● 4	● 0	● 3	● 1	● 0	Drydock
8.4	Generated project documentation with publishable rendering	● 5	● 1	● 1	● 2	● 1	Drydock
	Section average	4.8	0.5	2.5	1.8	0.3	

Read. The QuarterDeck is a first-class part of the methodology: markdown pages, editable specifications, a kanban of the Manifest, a Build Compass showing the costed work graph with buildable-now / review / done / failed badges, and questionnaires that feed the next command. Kiro's competing surface is genuinely good and better integrated with the editor, but it reviews the specification, not per-step build evidence. For a team that lives in an IDE the gap on 8.1 is narrower than the score suggests.

9 — Adoption Cost and Ecosystem

This is the section where Drydock loses, and it loses badly.

#	Capability	DD	SK	KI	BM	CM	Winner
9.1	Time from zero to first working code	● 1	● 3	● 4	● 2	● 5	CLAUDE.md
9.2	Right-sized for a small change or a one-file fix	● 1	● 2	● 3	● 3	● 5	CLAUDE.md
9.3	Zero install, no new tooling, no workspace concept	● 1	● 3	● 1	● 2	● 5	CLAUDE.md
9.4	Open source or no license cost	● 0	● 5	● 1	● 5	● 5	Tie: SK / BMAD / CLAUDE.md
9.5	Language and stack coverage out of the box	● 2	● 4	● 4	● 4	● 5	CLAUDE.md
9.6	Community, ecosystem, third-party extensions	● 1	● 5	● 3	● 4	● 5	Tie: Spec Kit / CLAUDE.md
	Section average	1.0	3.7	2.7	3.3	5.0	

Read. A plain CLAUDE.md sweeps this section 5.0 to 1.0, and that is not a rounding artifact — it is the strongest single result in the document. For anything under roughly a week of work the correct tool in this comparison is a rules file and a competent agent, and every ceremony Drydock imposes is pure loss. Drydock ships Python Rigging only, is proprietary, and has effectively no third-party ecosystem. Its `import → analyze → plan → build` chain is a long path to first running software, which is precisely the critique the literature levels at the whole category.

Aggregate — Five Products

Product	Sections 1-8: governed delivery	Section 9: adoption cost	All 45 rows
Drydock	● 4.6	● 1.0	4.11

Product	Sections 1-8: governed delivery	Section 9: adoption cost	All 45 rows
BMAD-METHOD	● 2.0	● 3.3	2.20
Kiro	● 1.7	● 2.7	1.82
Plain CLAUDE.md	● 0.9	● 5.0	1.47
GitHub Spec Kit	● 1.0	● 3.7	1.40

Read the two component columns, not the third. The result is a barbell: Drydock owns governed delivery by a factor of two over the nearest product and loses adoption cost by a factor of five to a text file. Everything in the middle is better than both at neither and adequate at some of each.

Do not read 4.11 as market dominance. Forty-five rows chosen from Drydock's feature space score Drydock highly by construction. Rerun the matrix with rows chosen by a two-person startup shipping weekly and CLAUDE.md wins outright.

Explicit Verdicts

Question	Winner	Margin
Turning an idea into a written specification	BMAD-METHOD	Clear. Drydock scores 0; it is an explicit non-goal.
Building working software <i>from</i> a specification	Drydock	Decisive — 4.2 against 2.5 next.
Proving the software is correct	Drydock	Decisive — 4.8 against 1.2 next; uniquely holds proof-integrity analysis.
Handling the second, third, and tenth change	Drydock	Decisive — 4.8 against 1.2 next.
Context and cost control	Drydock	Decisive on large work; ties CLAUDE.md on raw cost.
Human review surface	Drydock	Narrow over Kiro. Kiro integrates better; Drydock reviews more.
Parallel throughput	BMAD-METHOD	Weak win. Drydock scores 0 and should not.
Editor-native experience	Kiro	Uncontested. Drydock is a CLI plus a local web console.
Ecosystem and portability	GitHub Spec Kit	Clear. Ubiquitous, free, agent-neutral.
Cheapest, fastest, least ceremony	Plain CLAUDE.md	Overwhelming — 5.0 across the section.

Which To Use

Situation	Use	Why
One-file fix, small feature, exploratory hacking	Plain CLAUDE.md	Any process here costs more than the work.

Situation	Use	Why
Nothing written down; you need a product definition	BMAD-METHOD	Purpose-built elicitation; hand the output to a builder.
A free, agent-neutral team convention	GitHub Spec Kit	Lowest-friction shared vocabulary; accept that it drifts.
IDE-centric team, spec-first with editor hooks	Kiro	Best integrated authoring loop; watch metered inference cost.
Large system, specification already written, months of work	Drydock	Graph-ordered build, deterministic gates, evidence, refit.
Delivery that must be audited or defended	Drydock	The only product here producing per-step evidence and a release verdict with blockers.
Long-lived product whose specification keeps changing	Drydock	The only product here with drift detection, sealed specifications, and scoped replan.
Maximum agent throughput on independent work	None of these	BMAD is least bad; the field is immature.

The natural pairing is BMAD or Kiro upstream and Drydock downstream. They are not the same product and mostly do not compete. The one product Drydock genuinely must beat is the last row of Section 9 — a rules file and a competent agent — and it beats that only when the work is large enough that ordering, verification, and change control cost less than the chaos they replace. Below that threshold Drydock loses, and this document says so.

Part Two — The Wider Field

Eleven systems, scored on the same legend. Retained from the 2026-07-17 edition with Drydock's rows corrected to the current implementation.

Key	Product	License	Maturity level
DD	Drydock	Proprietary	Spec-anchored
SK	GitHub Spec Kit	Open source	Spec-first
KI	Kiro (AWS)	Proprietary IDE	Spec-first
TS	Tessl Framework + intent-integrity-kit	Proprietary	Spec-as-source
OS	OpenSpec	Open source	Spec-anchored
BM	BMAD-METHOD	Open source	Spec-first
KT	Spec Kitty	Open source	Spec-anchored
WA	Walden	Open source	Spec-anchored
SP	Superpowers	Open source	Spec-first
GS	GSD (Get Shit Done)	Open source	Spec-anchored
TR	Traycer	Proprietary	Spec-first

Maturity levels follow the taxonomy shared by Böckeler (martinfowler.com) and *Spec-Driven Development: From Code to Contract* (arXiv 2602.00180): **spec-first** (the specification guides the initial build, then drifts), **spec-anchored** (specification and code co-evolve, enforced by tests), **spec-as-source** (humans edit only specifications; code regenerates).

Matrix A — Drydock Core Features

The capabilities Drydock is built on. The field is scored against Drydock's own ground.

#	Feature	DD	SK	KI	TS	OS	BM	KT	WA	SP	GS	TR
A1	Typed Specification files with prescribed roles	5	3	3	4	3	3	3	3	1	3	2
A2	Dependency-graph build plan with runnable frontier	5	1	1	1	1	2	2	2	2	4	2
A3	Context-size-aware prompt stacking (token budget per step)	5	1	1	2	2	4	2	1	3	4	3
A4	LLM-estimated story points as token cost	5	0	0	0	0	2	0	0	0	1	0
A5	EARS-notation acceptance criteria, grammar-validated	5	0	2	0	0	0	0	5	0	0	0
A6	Guardrails as absolute prohibitions with a hard gate	5	2	0	2	0	0	1	3	0	2	0
A7	Evidence-bound scoring against Git HEAD and content hashes	5	0	0	3	0	0	1	4	1	2	0
A8	Discount for model-judged (non-deterministic) verification	5	0	0	2	0	0	0	2	0	0	0
A9	Blueprint drift detection via applied-spec hashing	4	0	0	4	2	0	1	5	0	2	1
A10	Sealed foundational specs requiring a change ticket	5	1	0	2	3	0	1	3	0	1	0
A11	Human review console for plan, evidence, and decisions	4	1	4	2	3	2	4	2	1	1	5
A12	Persistent intent injected into every prompt (Compass)	5	4	3	3	3	3	3	4	2	3	2
A13	Enterprise branding and stack rules injection (Rigging)	5	2	3	1	1	2	1	2	1	1	1
A14	Builder/user spec compaction to cut context cost	5	0	0	2	1	3	0	0	1	2	1
A15	Blocker/questionnaire loop that halts planning	5	1	2	1	2	3	2	3	3	3	2
A16	Brownfield reverse-engineering into specs	3	1	2	2	4	3	2	1	1	2	3
A17	Documentation generation and publishable rendering	5	1	1	2	1	2	1	1	0	1	1
A18	Subscription-CLI-only execution (no API-key spend)	5	3	0	0	3	3	3	4	4	3	0

#	Feature	DD	SK	KI	TS	OS	BM	KT	WA	SP	GS	TR
A19	Provider and IDE neutrality	4	5	0	2	4	4	5	5	2	2	3

Read. Drydock leads or ties on 16 of its own 19 core dimensions. The genuinely contested ground is A5 (Walden matches the EARS grammar validation), A9 (Walden's staleness chain is stricter), A11 (Traycer's review experience is more mature), and A19 (several open-source tools are more portable). A4, A8, A14, and A17 are effectively uncontested — nobody else prices stories in tokens, discounts model judgment, compacts specifications by audience, or ships a documentation pipeline.

Matrix B — Field Features Drydock Lacks or Underserves

Where the gaps live. These scores are the input to the backlog.

[illegible]

#	Feature	DD	SK	KI	TS	OS	BM	KT	WA	SP	GS	TR
B15	Dynamic model routing and cost tiering	1	0	0	1	0	2	1	0	2	5	3
B16	Gherkin or .feature executable behavioral specs	0	1	2	4	3	2	2	2	1	1	1
B17	Spec bloat and human-reviewability control	2	1	1	2	3	1	2	3	2	2	2
B18	Cross-repository specification sharing	0	0	0	4	3	1	0	0	0	0	0
B19	Bidirectional sync (fix the spec first, then regenerate code)	2	1	1	5	3	1	2	3	1	2	1
B20	Multi-agent role specialization	1	2	2	2	1	5	4	1	4	4	3

Changes since 2026-07-17. B3 rose from 1 to 4 on pre-build RED→GREEN enforcement. B5 rose from 0 to 5 — proof-integrity analysis now demotes vacuous, constant, and self-comparing proofs to UNVERIFIED, moving Drydock level with Tessl's intent-integrity-kit on the row that was previously the field's clearest advantage over Drydock. B14 rose from 0 to 4 on dependency legitimacy guardrails applied to generated Python dependency changes.

Read. Drydock's Matrix B average is now 1.45 against a field average near 1.8. Two of the three 2026-07 damage clusters have closed; one has not.

1. **Test integrity (B3, B4, B5) — largely closed.** Deterministic execution, vacuous-proof demotion, and RED→GREEN enforcement now combine with A8's discount on model judgment. The residual gap is B4: proofs are not hash-locked before implementation.
2. **Parallelism (B1, B2, B20) — open and unchanged.** Drydock still builds strictly serially down the frontier while Spec Kitty, Superpowers, and GSD execute independent work concurrently in isolated worktrees. Drydock already computes the graph that makes this safe.
3. **Right-sizing (B10, B11, B17) — open and unchanged.** Every critical source converges on the same complaint: SDD tools impose one heavyweight workflow regardless of problem size. Drydock is among the heaviest in the field. This is the most-cited failure mode in the literature and Drydock still has no answer.

Aggregate — Eleven Products

Product	Matrix A avg (Drydock's ground)	Matrix B avg (field's ground)	Combined
Drydock	4.74	1.45	3.10
GSD	1.95	2.85	2.40
Walden	2.63	1.85	2.24
Tessl + IKit	1.84	2.55	2.20
BMAD	1.89	1.80	1.85
OpenSpec	1.74	1.95	1.84

Product	Matrix A avg (Drydock's ground)	Matrix B avg (field's ground)	Combined
Spec Kitty	1.68	1.95	1.82
Superpowers	1.16	1.95	1.55
Traycer	1.37	1.55	1.46
GitHub Spec Kit	1.37	0.60	0.98
Kiro	1.16	0.60	0.88

Caveat. Matrix A is selected on Drydock's own feature set, so the 4.74 is partly definitional and is not market dominance. The honest signal is the delta: Drydock is far ahead on governance, context economics, and evidence, and behind the field's better tools on execution mechanics and workflow right-sizing.

Strategic Read

The defensible moat is the combination nobody else holds: token-priced story planning (A4) plus context-aware stacking (A3) plus audience-based compaction (A14) plus evidence-bound scoring that penalizes model judgment (A7, A8) — now reinforced by proof-integrity demotion (B5). Drydock is the only tool in the field that treats *context as an economic resource* and *model judgment as a liability to be discounted*. Nothing in the backlog should compromise that.

The nearest competitor is Walden, not Spec Kit or Kiro. Walden independently arrived at EARS validation, staleness chains, proof-based completion, and the same core thesis — that deterministic enforcement must be separated from non-deterministic drafting. Its advantages are portability (a single Go binary with no dependencies) and a CI release gate. Its disadvantages are no context economics, no story-point pricing, no compaction, and no review console.

The most dangerous critique is not a competitor. It is the shared conclusion of Böckeler and the Zenn skepticism piece that heavyweight SDD reintroduces waterfall by delaying feedback. The `import → analyze → quarterdeck → plan → build` chain is a long path to first working software. B10 and B11 are not nice-to-haves; they are the answer to the field's central objection, and they are the same finding Part One's Section 9 reaches from a completely different direction.

Where Drydock Must Improve

Ranked by the size of the honest loss, reconciling both parts.

1. **Right-sizing (9.1, 9.2, B10, B11, B17).** No lightweight path exists. A small change pays the full pipeline. Both halves of this analysis independently identify it as the top loss.
2. **Parallel execution (3.5, B1, B2).** Scoring zero while owning the dependency graph that makes parallelism safe is the clearest unforced loss in the document.

3. **Specification authoring (1.1).** Defensible as a non-goal, but it means Drydock cannot be a team's only tool, and every evaluation opens on an apparent zero.
4. **Stack coverage (9.5).** Python Rigging only. Every non-Python team must author governance before it can evaluate the product.
5. **Proof artifact locking (4.1, B4).** RED→GREEN is gated and vacuous proofs are demoted, but the proof is not hash-locked before implementation. Closing this makes verification unassailable.
6. **CI-portable release gate (B7).** The release verdict exists but is not packaged as an exit-code gate a pipeline can enforce. Walden leads this row with a 5.

Sources

- Böckeler, *Understanding Spec-Driven Development: Kiro, spec-kit, and Tessl* — <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- Fowler, *Structured-Prompt-Driven Development* — <https://martinfowler.com/articles/structured-prompt-driven/>
- *Spec-Driven Development: From Code to Contract* — <https://arxiv.org/html/2602.00180v1>
- cameronjsj/spec-compare, 13-tool feature matrix — <https://github.com/cameronjsj/spec-compare>
- specs.md compare — <https://specs.md/compare/overview>
- *The Spec: Living Specifications for Agentic Development* — <https://asdlc.io/patterns/the-spec/>
- intent-driven.dev best practices — <https://intent-driven.dev/knowledge/best-practices/>
- *Skepticism Toward Specification-Driven Development* — <https://zenn.dev/cbmrham/articles/202601-spec-driven-development-skepticism>
- GitHub Spec Kit — <https://github.com/github/spec-kit>
- Kiro — <https://kiro.dev/docs/>
- BMAD-METHOD — <https://docs.bmad-method.org/>
- OpenSpec — <https://github.com/Fission-AI/OpenSpec>
- Spec Kitty — <https://github.com/Priivacy-ai/spec-kitty>
- Walden — <https://andrearaponi.github.io/walden/>
- Superpowers — <https://github.com/obra/superpowers>
- GSD — <https://github.com/gsd-build/get-shit-done>
- Traycer — <https://docs.traycer.ai/quickstart>
- Tessl intent-integrity-kit — <https://tessl.io/registry/tessl-labs/intent-integrity-kit/2.7.5>